

06. Explicación profunda del código

PDF Reader v0.3.3 | Análisis 2026-09-29 | Referencia del release v0.3.3

Contenido

- Apertura y ciclo del documento
- Renderizado y zoom anclado
- Gestos
- Miniaturas y búsqueda
- Historial
- Electron
- Android
- Build y verificación

Apertura y ciclo del documento

`choosePdf` decide el adaptador. En Electron llama a `desktopPdf.pick`; en Web/Android abre el `<input type=file>`. `loadFile` rechaza una selección que no tenga extensión `.pdf` ni MIME `application/pdf`, lee el `ArrayBuffer` y delega a `loadPdfData`.

`loadPdfData` es el flujo central:

- 1 limpia el error y cambia a Lector;
- 2 destruye el `PDFDocumentProxy` previo, vacía caché/resultados y normaliza bytes a `Uint8Array`;
- 3 configura `PDF.js` con worker, CMaps, fuentes y WASM locales;
- 4 espera el documento y crea metadatos/identidad;
- 5 restaura página, zoom, ajuste y rotación dentro de rangos permitidos;
- 6 intenta guardar una copia en `IndexedDB`; si falla, avisa pero continúa;
- 7 renderiza la página actual y lanza miniaturas en segundo plano cooperativo.

Precondición: bytes disponibles en memoria. Poscondición exitosa: `state.doc` activo y canvas visible. Efectos secundarios: DOM, memoria, `localStorage` e `IndexedDB`. Un PDF de gran tamaño implica al menos una copia de bytes para el archivo de historial.

Renderizado y zoom anclado

`effectiveScale` obtiene el viewport a escala 1 con rotación combinada y calcula el espacio del visor. `width` usa ancho disponible, `page` el menor factor de ancho/alto y `manual` el zoom guardado. Todos los factores se limitan.

Antes de cambiar tamaño, `captureReadingAnchor` expresa el punto visible como proporción `(x,y)` del canvas y coordenada del viewport. Después, `restoreReadingAnchor` usa `readingAnchorDelta` para corregir `scrollLeft/scrollTop`. Esto evita saltos y mantiene accesibles ambos bordes.

`renderPage` incrementa `renderSeq`, calcula DPR con máximo 2, dimensiona tanto backing store como CSS, cancela el render anterior y espera `PDF.js`. Si otra llamada incrementó la secuencia, el resultado viejo ya no actualiza estado. Al finalizar guarda progreso. `RenderingCancelledException` se ignora deliberadamente; otros errores muestran una pantalla recuperable.

Gestos

`startTouch` distingue dos dedos de uno. La pinza guarda distancia inicial, escala, origen visual y ancla. `moveTouch` aplica solo una transformación CSS temporal, lo que mantiene respuesta fluida. `endTouch` convierte la razón a zoom real, elimina la transformación y renderiza. Con un dedo, un desplazamiento horizontal suficiente cambia página; un doble toque alterna ampliación/ancho. Caso delicado: el swipe no consulta si el canvas está desplazado horizontalmente, por lo que requiere pruebas físicas de interferencia.

Miniaturas y búsqueda

`buildThumbnails` crea un botón por página y renderiza canvas solo para las primeras 120. Cada seis iteraciones cede un frame. Esto limita parcialmente el costo visual pero no virtualiza la lista completa.

`pageText` extrae `textContent`, concatena cadenas y normaliza espacios; los resultados viven en `textCache` hasta cambiar de documento. `searchDocument` recorre todas las páginas secuencialmente y detecta una coincidencia por página. `buildSnippet` centra contexto; `highlightSnippet` escapa texto y consulta antes de insertar `<mark>` mediante `innerHTML`.

Historial

`database` abre `pdf-reader-local` versión 1 y crea `documents` con clave `id` e índice `lastOpened`. Cada operación abre una conexión, crea transacción, espera resultado y cierre. `saveHistoryDocument` hace `put`, abre otra transacción, ordena todas las filas y elimina las posteriores a la octava. No hay transacción única entre guardar y poder; un fallo intermedio puede dejar temporalmente más de ocho entradas.

La identidad usa nombre, tamaño y `lastModified`; no es un hash criptográfico. Dos documentos con los mismos metadatos pueden compartir progreso. En Android `lastModified` recibido por intent vale cero, aumentando esa posibilidad.

Electron

`main.cjs` solicita `single-instance lock`, detecta un argumento `.pdf`, crea una ventana aislada y deniega nuevas ventanas. `readPdf` valida solo extensión, obtiene metadatos y lee todo el archivo. Los handlers IPC exponen selección, lectura por ruta, inicio y configuración predeterminada. `preload.cjs` es la frontera: el renderer no obtiene `ipcRenderer` general.

`open-file` y `second-instance` envían la ruta al renderer. Inferencia basada en el código: el argumento inicial se localiza desde `process.argv[1..]`; opciones que terminen en `.pdf` podrían confundirse con rutas, aunque la lectura posterior fallaría de forma segura.

Android

`MainActivity` registra `PdfIntentPlugin` antes de `super.onCreate`. En un intent nuevo actualiza el intent actual y dispara `pdf-intent` en `WebView`.

`getPendingPdf` acepta solo `ACTION_VIEW` con URI y descarta el PDF especial de configuración. Lee por `ContentResolver` en bloques de 8192 bytes, corta sobre 128 MiB, codifica Base64 y consume el intent cambiándolo a `ACTION_MAIN`. Esta serialización crea copias adicionales en memoria. `requestDefault` genera un PDF temporal en caché, lo comparte mediante `FileProvider` y deja a Android la elección.

Build y verificación

`build-web` falla temprano si faltan los dos módulos `PDF.js`, recrea `dist` y copia recursos opcionales. `patch-android` toma versión y `versionCode` de `package.json`, reescribe Gradle/nombres y elimina cuatro permisos prohibidos. `verify-repo` actúa como prueba de contrato basada en archivos y marcadores. Es útil contra regresiones conocidas, pero no sustituye pruebas funcionales del DOM ni análisis del APK final.